



移动应用设计实践

FocusFlow 报告

学号: 1230205081

姓名: 聂志理

班级: 通信工程 232

浙江科技大学人工智能与信息工程学院

1. 目标

本项目旨在开发一款基于 Android 平台的专注效率工具——FocusFlow。该应用以任务为驱动核心，结合番茄钟计时法，帮助用户高效管理时间。主要目标包括：

- 提供快速任务创建与管理功能，降低操作摩擦
- 实现正向计时器，支持后台运行与前台服务通知
- 提供日/周/月多维度专注统计图表，可视化时间投入
- 支持深色模式、主题色切换、通知偏好等个性化设置

2. 设计描述

FocusFlow 是一款采用 MVVM 架构的 Android 原生应用，使用 Jetpack Compose + Material3 构建用户界面。应用以任务列表为首页，用户可通过键盘快速创建任务，点击「开始工作」进入专注计时。计时器使用 Foreground Service 保证后台持续运行，通知栏实时显示计时的任务标题与已用时间。

统计模块提供今日概览、日/周/月柱状图与折线图、任务耗时分布环形图，帮助用户回顾时间投入。设置模块支持亮色/暗色/跟随系统三种主题模式，6 种主题色，以及通知提示音与震动偏好。

3. 设计要求

- 开发语言：Java + Kotlin（混合开发）
- IDE：Android Studio
- SDK 版本：API 36
- 构建系统：Gradle 8.4 + AGP 8.3.0
- UI 框架：Jetpack Compose + Material3

4. 设计结构

应用采用 MVVM + Repository 架构，各层职责如下：

- 数据层 (data)：包含 Room 数据库 (Task、FocusSession 实体)、DAO 接口、Repository 仓库、DataStore 偏好存储。TaskDao 负责任务 CRUD 与搜索，FocusSessionDao 负责计时记录的聚合统计查询。
- UI 层 (ui)：采用 Jetpack Compose 声明式 UI，包含任务列表、计时器、统计、设置四大模块。每个模块对应一个 ViewModel (Java)，通过 LiveData 暴露状态驱动 UI 重组。
- 服务层 (service)：TimerService 继承 Service，以 Foreground Service 运行，使用 WakeLock 保持 CPU 活跃，通过通知栏展示计时进度并提供暂停/继续/停

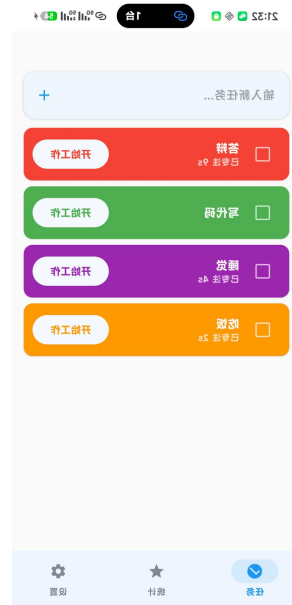
止操作。

- 工具层（util）：TimeFormatter 提供时间格式化工具，NotificationHelper 封装通知渠道创建与通知构建。

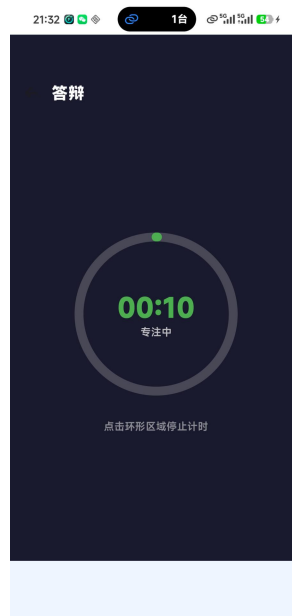
5. 界面设计

应用包含以下主要界面：

1. 任务列表页（首页）：顶部输入栏 + 任务卡片列表。每张卡片显示标题、累计专注时长、「开始工作」按钮。支持复选框标记完成、滑动删除。



2. 计时页：全屏环形进度条，环内显示已用时间与「专注中」标签。顶部返回箭头 + 任务标题。点击环形区域切换计时/停止。



3. 统计页：顶部今日概览卡片（总专注时长、番茄钟数、完成任务数、连续天数），日/周/月 Tab 切换柱状图/折线图，底部任务分布环形图。

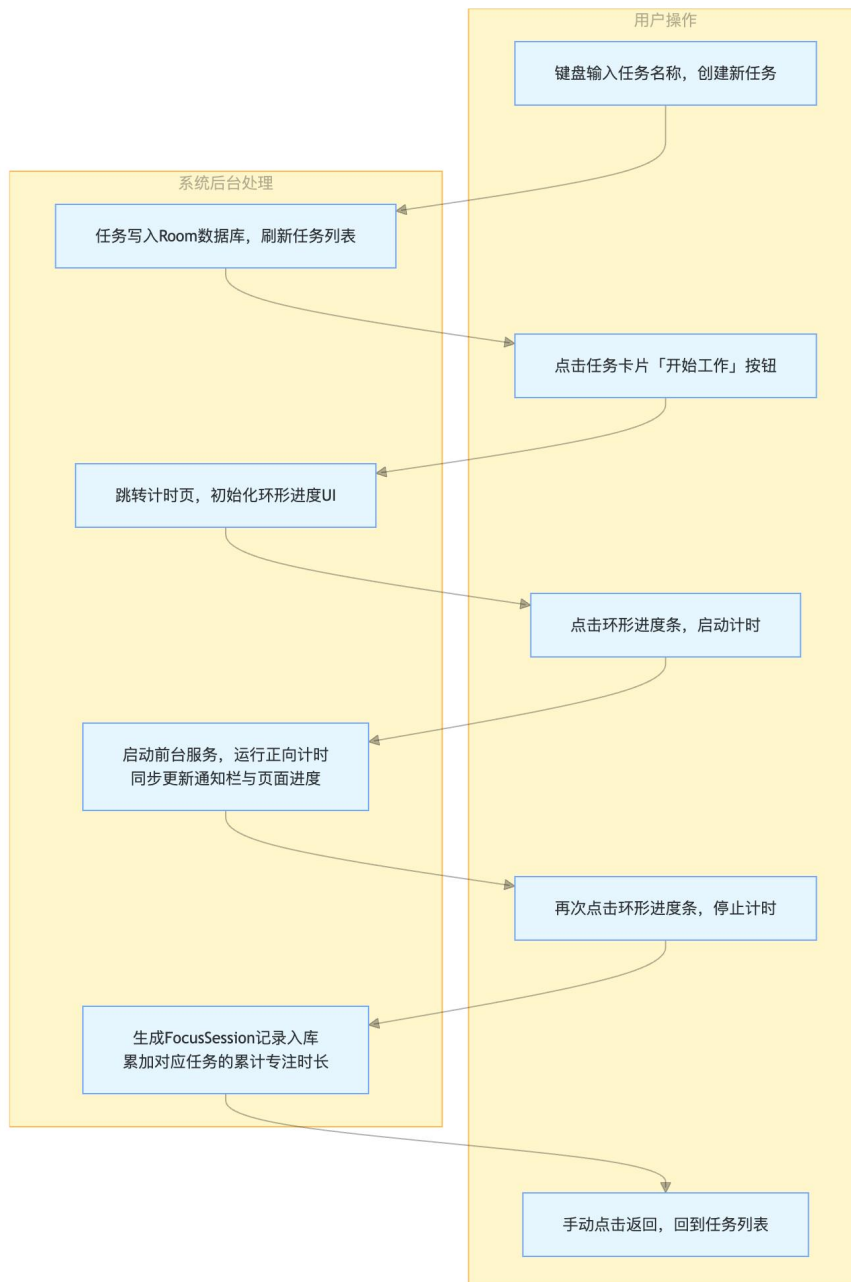


4. 设置页：主题模式选择（亮/暗/跟随系统）、主题色选择（6 种预设色）、通知开关、提示音选择（叮咚/铃声/风铃）、震动开关。

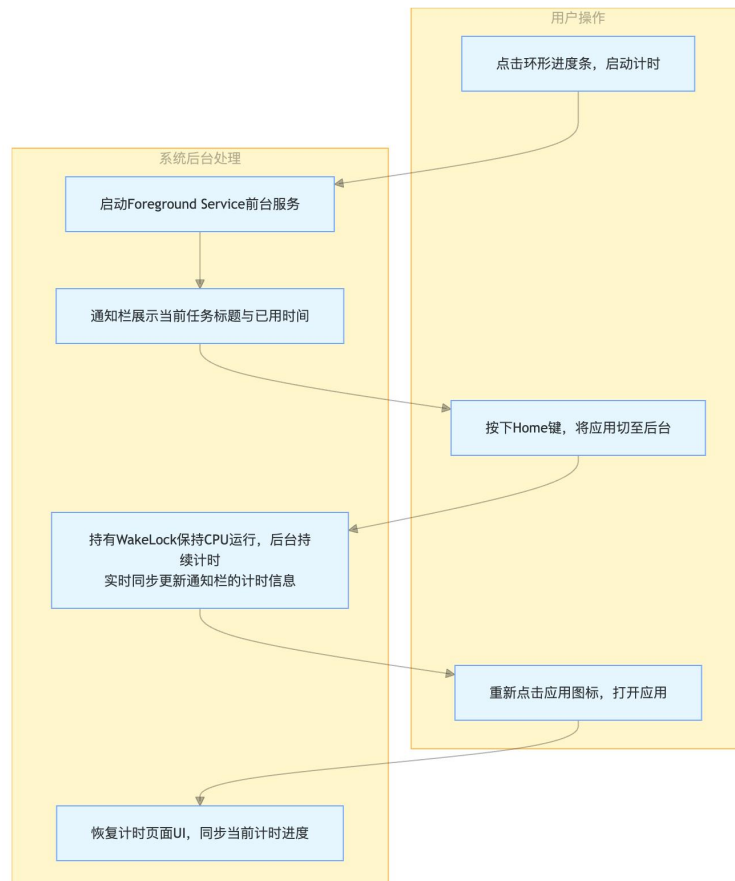


6. 主要功能流程图

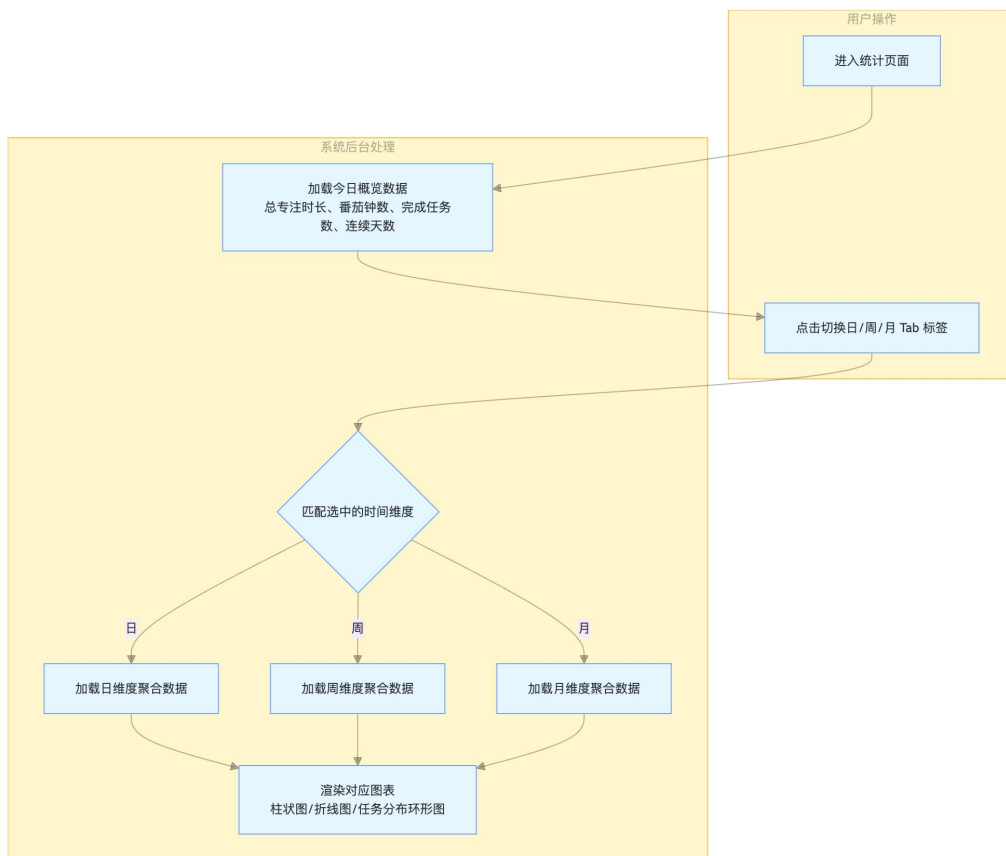
核心流程：键盘创建任务 → 任务列表 → 点击「开始工作」→ 进入计时页 → 点击环形进度条开始计时 → 再次点击停止计时 → 自动记录 FocusSession 并更新累计时长 → 手动返回任务列表



后台流程：开始计时 → 启动 Foreground Service → 通知栏显示任务标题与已用时间 → 按 Home 键切后台 → 计时继续 → 重新打开应用恢复计时状态



统计流程：进入统计页 → 加载今日概览 → 切换日/周/月 Tab → 加载对应聚合数据 → 渲染图表



6. 主要函数描述说明

- TaskDao.insert(Task task): 插入新任务到 Room 数据库，返回任务 ID
- TaskDao.getAllTasks(): 查询所有任务，按创建时间倒序排列，返回 LiveData<List<Task>>
- TaskDao.updateCompletion(long id, boolean isCompleted): 切换任务完成状态
- TaskDao.incrementFocusSeconds(long id, long seconds): 累加任务累计专注时长
- FocusSessionDao.insert(FocusSession session): 插入新的专注计时记录
- FocusSessionDao.getDailyStats(long startOfDay, long endOfDay): 获取每日统计数据（总时长、番茄钟数）
- FocusSessionDao.getWeeklyStats(long startOfWeek, long endOfWeek): 获取本周每日专注时长
- FocusSessionDao.getMonthlyStats(long startOfMonth, long endOfMonth): 获取本月每日专注时长
- FocusSessionDao.getTaskDistribution(): 获取各任务累计专注时长占比
- FocusSessionDao.getConsecutiveDays(long todayStart): 计算连续打卡天数
- TimerViewModel.startTimer(): 开始正向计时，启动 Observable.interval 定时器
- TimerViewModel.stopTimer(): 停止计时，创建 FocusSession，更新 Task.totalFocusSeconds
- TimerService.startForeground(): 启动前台服务，显示通知栏计时器

- UserPreferences.setThemeMode(int mode): 保存主题模式偏好到 DataStore
- UserPreferences.setThemeColor(int color): 保存主题色偏好到 DataStore

8. 文件结构

app/src/main/java/com/niezhili/focusflow/

```

├── MainActivity.java          # 入口 Activity, 请求通知权限
├── FocusFlowApp.java         # Application 子类, 初始化 Room +
DataStore + 通知渠道
├── ComposeEntry.kt           # Kotlin 桥接, 设置 ComposeContent
├── data/
│   ├── local/AppDatabase.java # Room 数据库单例
│   ├── entity/Task.java       # 任务实体
│   ├── entity/FocusSession.java # 专注记录实体
│   ├── dao/TaskDao.java       # 任务 DAO
│   ├── dao/FocusSessionDao.java # 专注记录 DAO
│   ├── dao/DailyStats.java    # 统计 POJO
│   ├── dao/TaskDistribution.java # 任务分布 POJO
│   ├── repository/TaskRepository.java
│   ├── repository/FocusRepository.java
│   └── preferences/UserPreferences.java
├── ui/
│   ├── navigation/AppNavigation.kt
│   ├── MainScreen.kt
│   ├── theme/{Color,Theme,Type}.kt
│   ├── task/{TaskListScreen,TaskListViewModel}.java + components/
│   ├── timer/{TimerScreen,TimerViewModel}.java + components/
│   ├── stats/{StatsScreen,StatsViewModel}.java + components/
│   └── settings/{SettingsScreen,SettingsViewModel}.java
├── service/TimerService.java  # 前台计时服务
└── util/{TimeFormatter,NotificationHelper}.java

```


9. App 运行截图

